Vityarthi Project
Of
Fundamentals Of AI And ML

Project of

Varun Rohila : 25BAS10048

Course Instructor : J. MANIKANDAN
Class no: BL025260501072
Slot: B11+B12+B13

VIT Bhopal University

Bhopal - Indore Highway

Kothri Kalan, Sehore - 466114

Madhya Pradesh

1. Introduction and Problem Statement

In the fields of aerospace engineering and robotics, identifying the root cause of a test flight failure is a difficult and time-consuming task. During a test flight of a UAV (Unmanned Aerial Vehicle) or drone, the onboard flight controller generates massive CSV log files containing thousands of rows of raw telemetry data per minute. This data includes critical parameters such as battery voltage, altitude, pitch angle, and wind resistance.

For student design teams or hobbyists, manually scanning these massive datasets to pinpoint the exact millisecond a battery voltage dropped or wind shear caused a severe pitch angle is highly inefficient and prone to human error. I chose to tackle this problem for my BYOP submission because it bridges the gap between raw hardware data and actionable insights. The problem is a real-world bottleneck in prototyping, and solving it requires structured data parsing and algorithmic anomaly detection.

telemetry.csv

```
Time,Altitude_m,Voltage_V,Pitch_deg,Wind_ms
00:01,0,16.4,0,2
00:02,10,16.2,5,3
00:03,45,16.0,12,5
00:04,120,15.8,10,18
00:05,125,15.5,-25,22
00:06,110,13.2,-15,10
00:07,50,13.0,0,5
00:08,0,12.8,0,2
```

2. Proposed Solution and Relevance to Course

To solve this, I built a localized Python data pipeline and analysis tool. The core objective was to treat flight logs as tabular data and use data processing concepts learned in this course to identify critical outliers.

Rather than relying on manual inspection, the system automatically ingests the CSV file, iterates through the flight data, and flags specific rows that violate predefined safety thresholds (e.g., critical voltage drops or dangerous pitch instability). The system then aggregates these anomalies and outputs a formatted "Post-Flight ML Debrief" directly into the terminal, pointing the user to the exact timestamp of the failure.

3. Technical Approach and Key Decisions

The project was developed using Python 3.9 within the Visual Studio Code (VS Code) environment.

Key Technical Decisions:

- **Useof PandasforDataHandling:** I utilized the pandas library to ingest and structure the raw CSV data into a DataFrame. This allowed for efficient parsing and manipulation of the tabular data compared to standard Python list iterations.
- **CLI (Command Line Interface) Architecture:** I decided to keep the application entirely terminal-based. Building a complex Graphical User Interface (GUI) would have distracted from the core data processing logic. A CLI ensures the tool is fast, lightweight, and can easily be run on a low-end laptop out in a testing field.
- **Algorithmic Thresholding:** Instead of a black-box approach, the anomaly detection relies on strict, programmable parameters based on actual physics (e.g., flagging a voltage drop below 14.0V or a pitch angle exceeding 20 degrees).

analyzer.py

```python
import pandas as pd
from sklearn.ensemble import IsolationForest
import warnings

# Suppress warnings for cleaner terminal output
warnings.filterwarnings('ignore')

def train_and_detect_anomalies(csv_file):
    """
    Uses an Unsupervised Machine Learning model (Isolation Forest)
    to detect abnormal behavior in UAV flight data.
    """
    print(f"Loading flight data from {csv_file}...\n")
    try:
        df = pd.read_csv(csv_file)
    except FileNotFoundError:
        print("Error: Could not find telemetry.csv")
        return

    # Select the physical metrics the ML model should look at
    features = ['Altitude_m', 'Voltage_V', 'Pitch_deg', 'Wind_ms']
    X = df[features]

    print("Training pure Machine Learning Model on flight data...")

    # Initialize the Machine Learning Model
    model = IsolationForest(contamination=0.25, random_state=42)

    # Train the model and make predictions
```

4. Challenges Faced and Debugging

Developing this pipeline presented several challenges, primarily related to environment configuration and data validation.

- **Dependency and Path Management:** While setting up the environment on macOS, I encountered PATHconfiguration warnings when installing pandas, numpy, and scikit-learn via pip3. This required careful terminal management to ensure the script was executing using the correct Python installation and that all libraries were properly linked.
- **Data Validation Errors (EmptyDataError):** During early testing, the script crashed entirely, throwing a pandas.errors.EmptyDataError: No columns to parse from file. Upon debugging, I discovered this occurred because my test telemetry.csv file was completely empty.

This error taught me a valuable lesson in building robust data pipelines: a system must first validate that the data exists and is properly formatted before attempting to parse it, otherwise, the entire application will crash.

5. Final Results and Conclusion

After resolving the data ingestion issues, the script successfully executed. It accurately read the populated telemetry.csv file, processed the parameters against the safety thresholds, and successfully printed the Post-Flight Debrief. The output correctly highlighted two specific anomalies at timestamp and, noting extreme wind and voltage drops.

```
varunrohila@Varuns-MacBook-Air uav-telemetry-analyzer % python3 analyzer.py
Training pure Machine Learning Model on flight data...
--------------------------------------------------
🚨 POST-FLIGHT ML DEBRIEF: ANOMALIES DETECTED
--------------------------------------------------
CRITICAL WARNING: 2 anomalies detected by ML algorithm!

Timestamp: [00:05]
  -> Data Signature: Alt: 125m | Volts: 15.5V | Pitch: -25° | Wind: 22m/s
  -> ML Assessment: Data point deviates significantly from normal flight parameters.

Timestamp: [00:06]
  -> Data Signature: Alt: 110m | Volts: 13.2V | Pitch: -15° | Wind: 10m/s
  -> ML Assessment: Data point deviates significantly from normal flight parameters.

varunrohila@Varuns-MacBook-Air uav-telemetry-analyzer %
```

This project solidified my understanding of handling messy, real-world tabular data using Python. I learned how to isolate specific features from a larger dataset and flag them based on logical conditions. Most importantly, I learned that the value of an analytical tool lies not just in finding the error, but in how clearly it communicates that error back to the engineer so they can fix the hardware.